

SLAM : Projet Agora Sprint 5 Demande 2**Générer des formulaires CRUD
avec le composant Doctrine****Auteur :** Yannick Narbey**Pré-requis :** Symfony : contrôleurs, routes, templates twig**Versions minimum à respecter :**

- Apache 2.4.54.2, MySQL 8.0.31, PHP 8.2.0 (PHP 7.4 provoque une erreur lors de l'utilisation de la commande make)
- Symfony CLI 5.5.10, Symfony 6.3, composer 2.6.4 (*composer self-update pour actualiser composer*)

T A B L E D E S M A T I E R E S

1.	Contexte et objectifs	2
2.	Mise en place des prérequis techniques	Erreur ! Signet non défini.
2.1	Installation des modules Symfony complémentaires	Erreur ! Signet non défini.
2.2	Configuration et création de la nouvelle base de données	Erreur ! Signet non défini.
3.	Création de l'entité et de la table Genre	Erreur ! Signet non défini.
3.1	Point sur l'architecture du projet	Erreur ! Signet non défini.
3.2	Création de l'entité genre	Erreur ! Signet non défini.
3.3	Création de la table genre	Erreur ! Signet non défini.
4.	Génération du formulaire CRUD de gestion des genres	3
4.1	Installation des paquets nécessaires à la génération des formulaires CRUD	3
4.2	Génération des formulaires	4
4.3	Génération des routes	6
4.4	Génération des contrôleurs et vues	6
4.5	Tests	10
4.6	Ajustement du menu pour appeler le nouveau controleur	12
5.	Modification du style des nouveaux formulaires	15
5.1	Modification du template genre/index.html.twig :	15
5.2	Modification du template genre/edit.html.twig :	17
5.3	Modification du template genre/_delete_form.html.twig :	18
5.4	Modification du template genre/new.html.twig :	19
5.5	Modification du template genre/_form.html.twig :	20
5.6	Modification du template genre/show.html.twig :	21
5.7	Suppression de l'ancien CRUD de gestion des genres	24
6.	A vous de jouer	27

1. Contexte et objectifs

L'association Agora souhaite, dans le sprint suivant, pouvoir donner des **droits d'accès différents aux utilisateurs** en fonction de leur rôle dans l'association et cela au travers d'une **interface pour gérer les comptes d'accès**.

L'administration des données de l'application doit donc évoluer à terme pour s'appuyer sur les mécanismes de sécurité natifs de Symfony.

Pour cela, il devient nécessaire de reconstruire l'ensemble des formulaires à l'aide de l'ORM Doctrine qui seront pris en charge nativement par le composant de sécurité de Symfony.

L'ESN Logma souhaite utiliser les fonctionnalités du framework Symfony pour automatiser la création de ces formulaires de gestion.



Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)

Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.

SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.



<https://symfony.com/>



Doctrine

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.



<https://www.doctrine-project.org>

Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

2. Génération du formulaire CRUD de gestion des genres

A ce point du projet, la nouvelle base de données agoraorm contient la table genre qui est gérable au travers d'un contrôleur créé de toutes pièces au cours de la précédente étape.

L'objectif va donc être de reconstruire des formulaires de gestion de l'application, en tirant partie des fonctionnalités de l'ORM. Dans un premier temps, nous allons reconstruire les formulaires CRUD les plus simples, soit :

- Gérer les genres,
- Gérer les pegis,
- Gérer les plateformes,
- Gérer les marques.

2.1 Installation des paquets nécessaires à la génération des formulaires CRUD

C:\wamp64\www\agoram5>composer require form validator twig-bundle security-csrf

```
C:\Windows\System32\cmd.exe x + v
C:\wamp64\www\agoram5>composer require form validator twig-bundle security-csrf
./composer.json has been updated
Running composer update symfony/form symfony/validator symfony/twig-bundle symfony/security-csrf
Loading composer repositories with package information
Restricting packages listed in "symfony/symfony" to "6.0.*"
Updating dependencies
Lock file operations: 9 installs, 0 updates, 0 removals
- Locking symfony/form (v6.0.19)
- Locking symfony/options-resolver (v6.0.19)
- Locking symfony/password-hasher (v6.0.19)
- Locking symfony/polyfill-intl-icu (v1.33.0)
- Locking symfony/property-access (v6.0.19)
- Locking symfony/property-info (v6.0.19)
- Locking symfony/security-core (v6.0.19)
- Locking symfony/security-csrf (v6.0.19)
- Locking symfony/validator (v6.0.19)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 9 installs, 0 updates, 0 removals
- Downloading symfony/property-info (v6.0.19)
- Downloading symfony/property-access (v6.0.19)
- Downloading symfony/options-resolver (v6.0.19)
- Downloading symfony/form (v6.0.19)
- Downloading symfony/password-hasher (v6.0.19)
- Downloading symfony/security-core (v6.0.19)
- Downloading symfony/security-csrf (v6.0.19)
- Downloading symfony/validator (v6.0.19)
- Installing symfony/property-info (v6.0.19): Extracting archive
- Installing symfony/property-access (v6.0.19): Extracting archive
- Installing symfony/polyfill-intl-icu (v1.33.0): Extracting archive
- Installing symfony/options-resolver (v6.0.19): Extracting archive
- Installing symfony/form (v6.0.19): Extracting archive
- Installing symfony/password-hasher (v6.0.19): Extracting archive
- Installing symfony/security-core (v6.0.19): Extracting archive
- Installing symfony/security-csrf (v6.0.19): Extracting archive
- Installing symfony/validator (v6.0.19): Extracting archive
Package doctrine/cache is abandoned, you should avoid using it. No replacement was suggested.
Package sensio/framework-extra-bundle is abandoned, you should avoid using it. Use Symfony instead.
Generating optimized autoload files
Class PdoJeux located in ./src/Controller/modele/class.PdoJeux.inc.php does not comply with psr-4 autoloading standard (rule: App\ => ./src). Skipping.
65 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
Symfony operations: 1 recipe (9d24302ab037887d710c043c3f855cdf)
- Configuring symfony/validator (>=5.3): From github.com/symfony/recipes:main
Executing script cache:clear [OK]
Executing script assets:install public [OK]

What's next?

Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.

Found 8 security vulnerability advisories affecting 6 packages.
Run "composer audit" for a full list of advisories.
```

Cela va ajouter :

- symfony/form → pour générer des formulaires

- symfony/validator → pour valider les données automatiquement
- symfony/twig-bundle → pour générer les vues Twig
- symfony/security-csrf → pour protéger les formulaires (anti-CSRF)

Editer le fichier composer.json et vérifier que ces nouveaux packages ont été ajoutés au projet :

```

1  {
2      "type": "project",
3      "license": "proprietary",
4      "minimum-stability": "stable",
5      "prefer-stable": true,
6      "require": {
7          "php": ">=8.0.2",
8          "ext-ctype": "*",
9          "ext-iconv": "*",
10         "doctrine/dbal": "^3",
11         "doctrine/doctrine-bundle": "^2.13",
12         "doctrine/doctrine-migrations-bundle": "^3.4",
13         "doctrine/orm": "^2.20",
14         "sensio/framework-extra-bundle": "^6.2",
15         "symfony/asset": "6.0.*",
16         "symfony/console": "6.0.*",
17         "symfony/dotenv": "6.0.*",
18         "symfony/flex": "^2",
19         "symfony/form": "6.0.*",
20         "symfony/framework-bundle": "6.0.*",
21         "symfony/monolog-bundle": "^3.8",
22         "symfony/runtime": "6.0.*",
23         "symfony/security-csrf": "6.0.*",
24         "symfony/twig-bundle": "6.0.*",
25         "symfony/validator": "6.0.*",
26         "symfony/yaml": "6.0.*"
27     },

```

2.2 Suppression du contrôleur créé au cours de la précédente étape

Un contrôleur GenreControleur.php a été créé manuellement au cours de l'étape précédente. Cela a permis d'appréhender le rôle de chacune des routes (création, modification, lecture, suppression) des relations entre le contrôleur, la classe entité, le repository.

Maintenant que cette étape est maîtrisée, nous allons utiliser des fonctionnalités mise à disposition par le framework Symfony pour générer automatiquement ces fichiers mais également les formulaires.

Arrêter le serveur à l'aide de la combinaison de touches ctrl+C :

```

M genre"
[Application] Nov  1 03:30:38 |INFO   | DOCTRI Disconnecting

Shutting down! Waiting for all workers to be done.
[PHP-CGI    ] Nov  1 04:59:12 |INFO   | RUNNER Signal "interrupt" received, forwarding and exiting
cmd="PHP-CGI"

remove C:\Users\narbe\.symfony5\log\f8a5cdd00ff7a294e6770b5339210dc715d72f5f.log: Le processus ne
peut pas accéder au fichier car ce fichier est utilisé par un autre processus.

C:\wamp64\www\agoram50>

```

Supprimer :

- le fichier contrôleur /src/Controller/GenreController.php

- Le dossier de vue templates/genre

Veillez à conserver les fichiers entity /src/Entity/Genre.php et /src/Repository/GenreRepository.php.

Ces fichiers et ce répertoire vont être recréés automatiquement sur la base de l'entité, qu'il faut donc conserver, en même temps que les formulaires CRUD.

Editer le fichier menu.html.twig dans le répertoire templates et remplacer app_genre par app_genre_index:

```

1  <!--
2  *****
3  MAIN SIDEBAR MENU
4  *****
5  -->
6  <!--sidebar start-->
7  <aside>
8  <div id="sidebar" class="nav-collapse">
9      <!-- sidebar menu start-->
10     <ul class="sidebar-menu" id="nav-accordion">
11         <p class="centered"><a href="{path('accueil')}">MJC Agora</h5>
13         <li class="sub-menu">
14             <a{% if menuActif is defined and menuActif == 'Jeux' %} class="active" {% endif
15                 <i class="fa fa-gamepad"></i>
16                 <span>Jeux vidéos</span>
17             </a>
18             <ul class="sub">
19                 <li><a href="{path('jeux_afficher')}">Jeux</a></li>
20                 <li><a href="{path('genres_afficher')}">Genres</a></li>
21                 <li><a href="{path('app_genre_index')}">Nouvelle page Genres</a></li>
22                 <li><a href="{path('plateformes_afficher')}">Plateformes</a></li>
23                 <li><a href="{path('marques_afficher')}">Marques</a></li>
24                 <li><a href="{path('pegis_afficher')}">Pegi</a></li>
25             </ul>

```

Le formulaire qui va être généré automatiquement a pour route app_genre_index et pas app_genre.

2.3 Génération des formulaires

Passer la commande suivante pour que Symfony génère automatiquement le(s) formulaire(s) CRUD de gestion des genres :

C:\wamp64\www\agoram5>php bin/console make:crud Genre

```

C:\wamp64\www\agoram5test>php bin/console make:crud Genre

Choose a name for your controller class (e.g. GenreController) [GenreController]:
>

Do you want to generate tests for the controller?. [Experimental] (yes/no) [no]:
>

created: src/Controller/GenreController.php
created: src/Form/GenreType.php
created: templates/genre/_delete_form.html.twig
created: templates/genre/_form.html.twig
created: templates/genre/edit.html.twig
created: templates/genre/index.html.twig
created: templates/genre/new.html.twig
created: templates/genre/show.html.twig

Success!

Next: Check your new CRUD by going to /genre/

C:\wamp64\www\agoram5test>

```

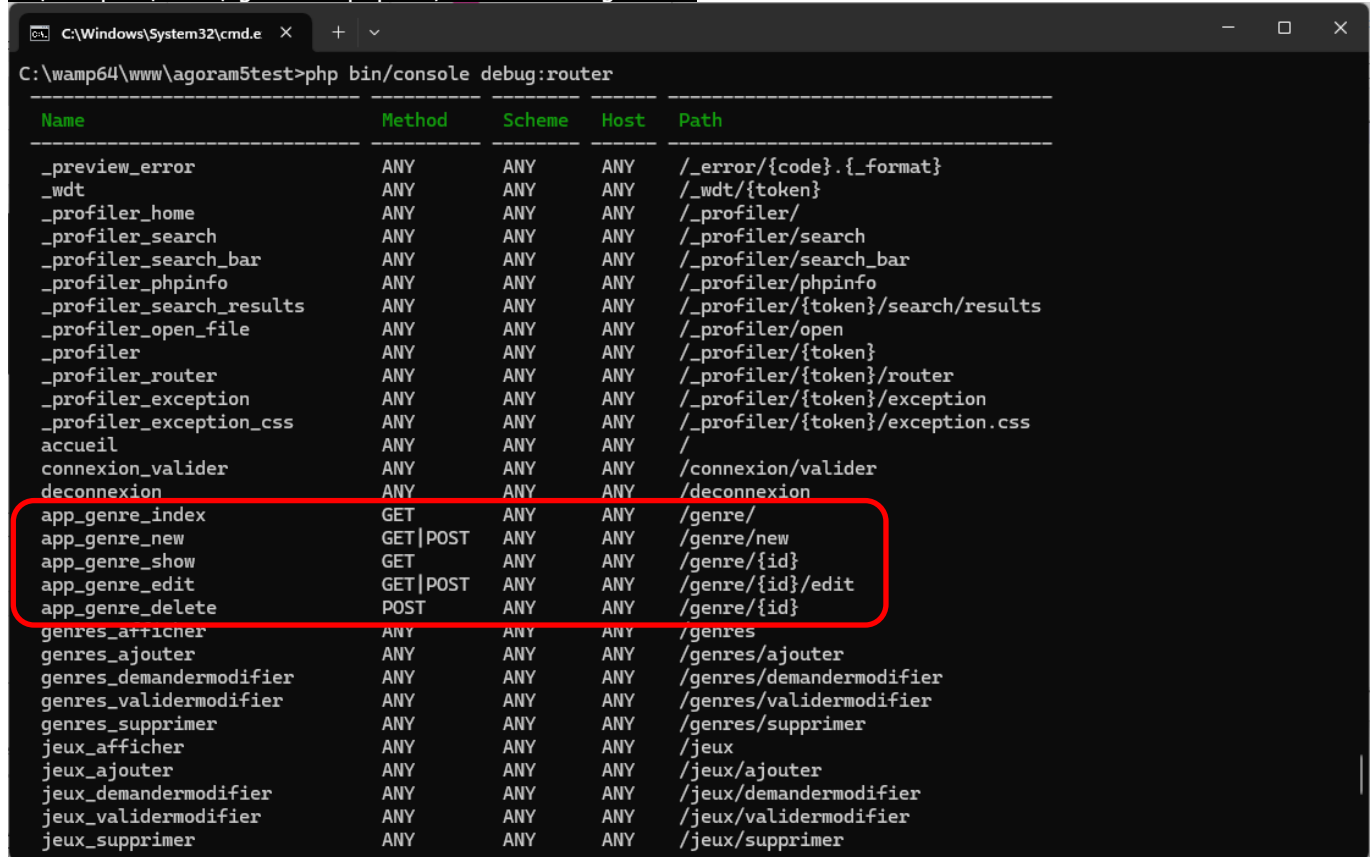
2.4 Génération des routes

Symfony va générer les routes, dont la principale :

<http://localhost:8000/genre>

Lancer la commande suivante pour vérifier que ces routes ont bien été générées, faire la distinction entre les routes issues du sprint précédent et les nouvelles « app_genre_... » générée avec la commande « php bin/console make:crud GenreGenre ».

C:\wamp64\www\agoram5>php bin/console debug:router



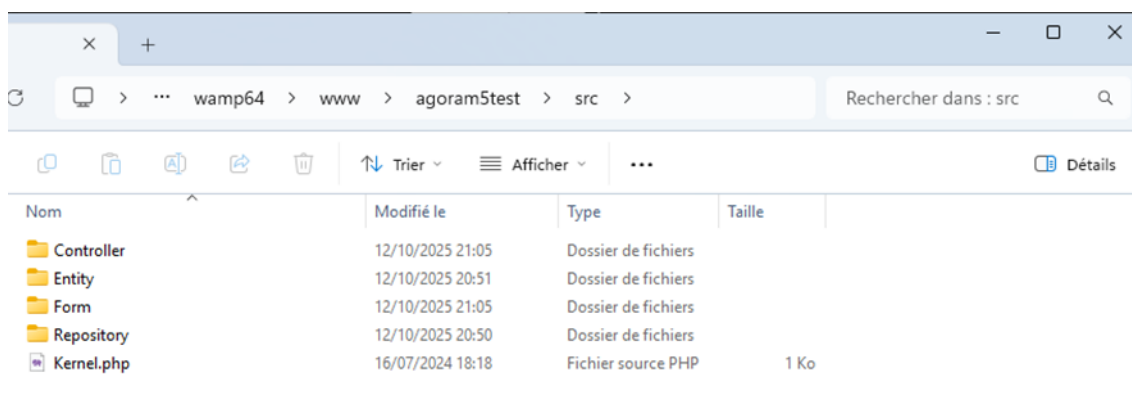
Name	Method	Scheme	Host	Path
_preview_error	ANY	ANY	ANY	/_error/{code}.{_format}
_wdt	ANY	ANY	ANY	/_wdt/{token}
_profiler_home	ANY	ANY	ANY	/_profiler/
_profiler_search	ANY	ANY	ANY	/_profiler/search
_profiler_search_bar	ANY	ANY	ANY	/_profiler/search_bar
_profiler_phpinfo	ANY	ANY	ANY	/_profiler/phpinfo
_profiler_search_results	ANY	ANY	ANY	/_profiler/{token}/search/results
_profiler_open_file	ANY	ANY	ANY	/_profiler/open
_profiler	ANY	ANY	ANY	/_profiler/{token}
_profiler_router	ANY	ANY	ANY	/_profiler/{token}/router
_profiler_exception	ANY	ANY	ANY	/_profiler/{token}/exception
_profiler_exception_css	ANY	ANY	ANY	/_profiler/{token}/exception.css
accueil	ANY	ANY	ANY	/
connexion_valider	ANY	ANY	ANY	/connexion/valider
deconnexion	ANY	ANY	ANY	/deconnexion
app_genre_index	GET	ANY	ANY	/genre/
app_genre_new	GET POST	ANY	ANY	/genre/new
app_genre_show	GET	ANY	ANY	/genre/{id}
app_genre_edit	GET POST	ANY	ANY	/genre/{id}/edit
app_genre_delete	POST	ANY	ANY	/genre/{id}
genres_attacher	ANY	ANY	ANY	/genres
genresajouter	ANY	ANY	ANY	/genres/ajouter
genresdemandermodifier	ANY	ANY	ANY	/genres/demandermodifier
genresvalidermodifier	ANY	ANY	ANY	/genres/validermodifier
genres_supprimer	ANY	ANY	ANY	/genres/supprimer
jeux_afficher	ANY	ANY	ANY	/jeux
jeuxajouter	ANY	ANY	ANY	/jeux/ajouter
jeuxdemandermodifier	ANY	ANY	ANY	/jeux/demandermodifier
jeuxvalidermodifier	ANY	ANY	ANY	/jeux/validermodifier
jeux_supprimer	ANY	ANY	ANY	/jeux/supprimer

2.5 Génération des contrôleurs et vues

Symfony va générer les vues et les formulaires aux travers des fichiers suivants :

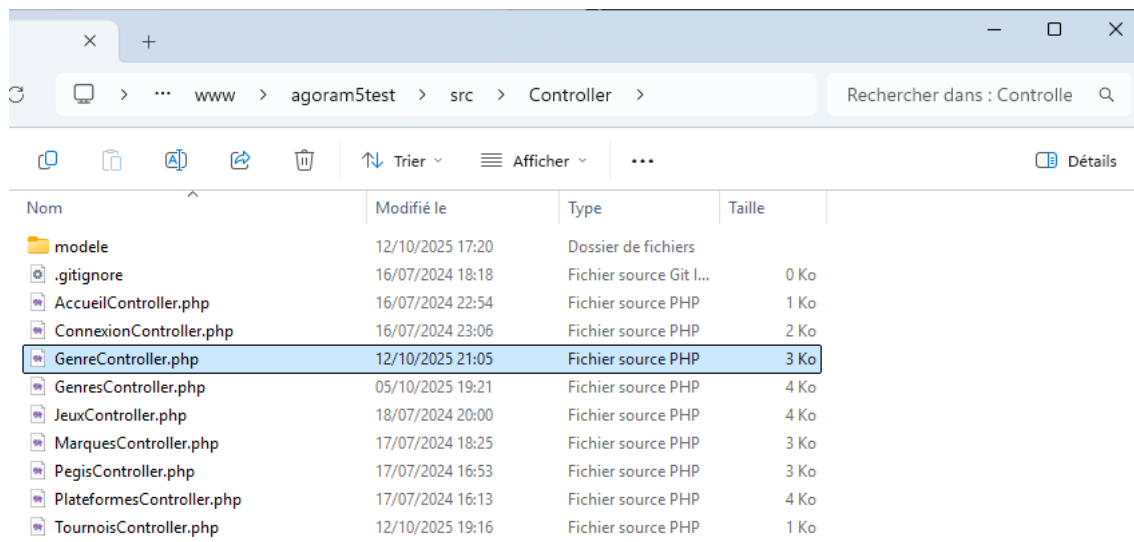
- src/Controller/GenreController.php
- src/Form/GenreType.php
- templates/genre/ ...html.twig

Vérifier la création du dossier Controller qui contient le controleur qui pilote les vues du CRUD permettant de gérer les genres :



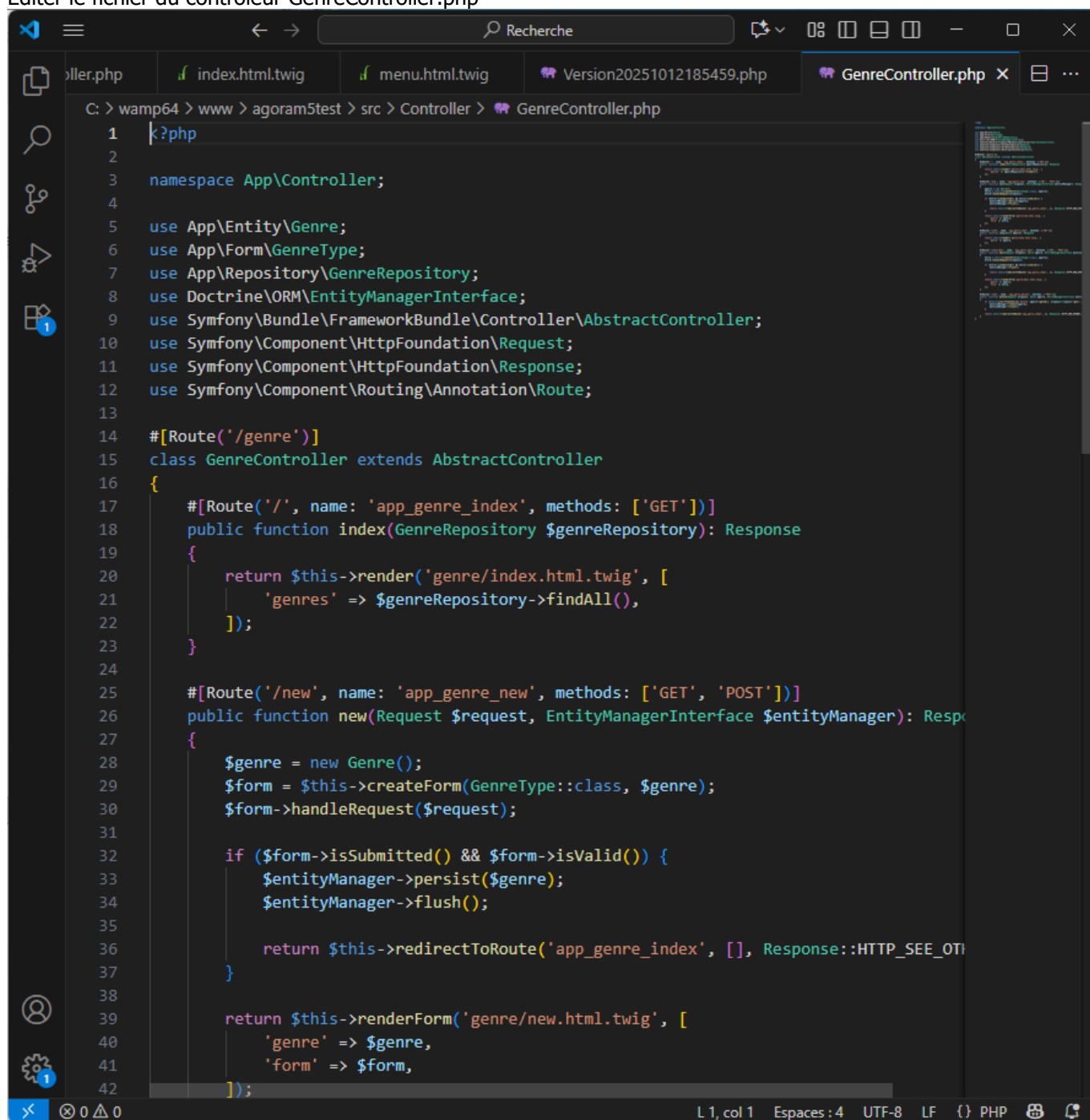
Nom	Modifié le	Type	Taille
Controller	12/10/2025 21:05	Dossier de fichiers	
Entity	12/10/2025 20:51	Dossier de fichiers	
Form	12/10/2025 21:05	Dossier de fichiers	
Repository	12/10/2025 20:50	Dossier de fichiers	
Kernel.php	16/07/2024 18:18	Fichier source PHP	1 Ko

Le controleur GenreController.php, cohabite avec l'ancien controleur GenresController.php que nous allons supprimer.



Nom	Modifié le	Type	Taille
modele	12/10/2025 17:20	Dossier de fichiers	
.gitignore	16/07/2024 18:18	Fichier source Git I...	0 Ko
AccueilController.php	16/07/2024 22:54	Fichier source PHP	1 Ko
ConnexionController.php	16/07/2024 23:06	Fichier source PHP	2 Ko
GenreController.php	12/10/2025 21:05	Fichier source PHP	3 Ko
GenresController.php	05/10/2025 19:21	Fichier source PHP	4 Ko
JeuxController.php	18/07/2024 20:00	Fichier source PHP	4 Ko
MarquesController.php	17/07/2024 18:25	Fichier source PHP	3 Ko
PegisController.php	17/07/2024 16:53	Fichier source PHP	3 Ko
PlateformesController.php	17/07/2024 16:13	Fichier source PHP	4 Ko
TournoisController.php	12/10/2025 19:16	Fichier source PHP	1 Ko

Editer le fichier du controleur GenreController.php

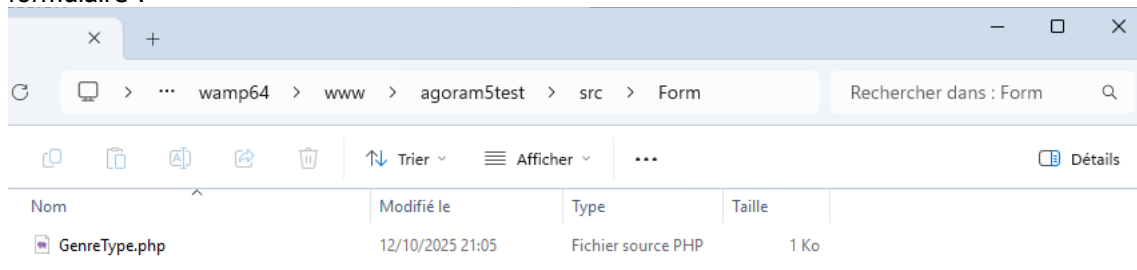


```
1 k?php
2
3 namespace App\Controller;
4
5 use App\Entity\Genre;
6 use App\Form\GenreType;
7 use App\Repository\GenreRepository;
8 use Doctrine\ORM\EntityManagerInterface;
9 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
10 use Symfony\Component\HttpFoundation\Request;
11 use Symfony\Component\HttpFoundation\Response;
12 use Symfony\Component\Routing\Annotation\Route;
13
14 #[Route('/genre')]
15 class GenreController extends AbstractController
16 {
17     #[Route('/', name: 'app_genre_index', methods: ['GET'])]
18     public function index(GenreRepository $genreRepository): Response
19     {
20         return $this->render('genre/index.html.twig', [
21             'genres' => $genreRepository->findAll(),
22         ]);
23     }
24
25     #[Route('/new', name: 'app_genre_new', methods: ['GET', 'POST'])]
26     public function new(Request $request, EntityManagerInterface $entityManager): Response
27     {
28         $genre = new Genre();
29         $form = $this->createForm(GenreType::class, $genre);
30         $form->handleRequest($request);
31
32         if ($form->isSubmitted() && $form->isValid()) {
33             $entityManager->persist($genre);
34             $entityManager->flush();
35
36             return $this->redirectToRoute('app_genre_index', [], Response::HTTP_SEE_OTHER);
37         }
38
39         return $this->renderForm('genre/new.html.twig', [
40             'genre' => $genre,
41             'form' => $form,
42         ]);
43     }
44 }
```

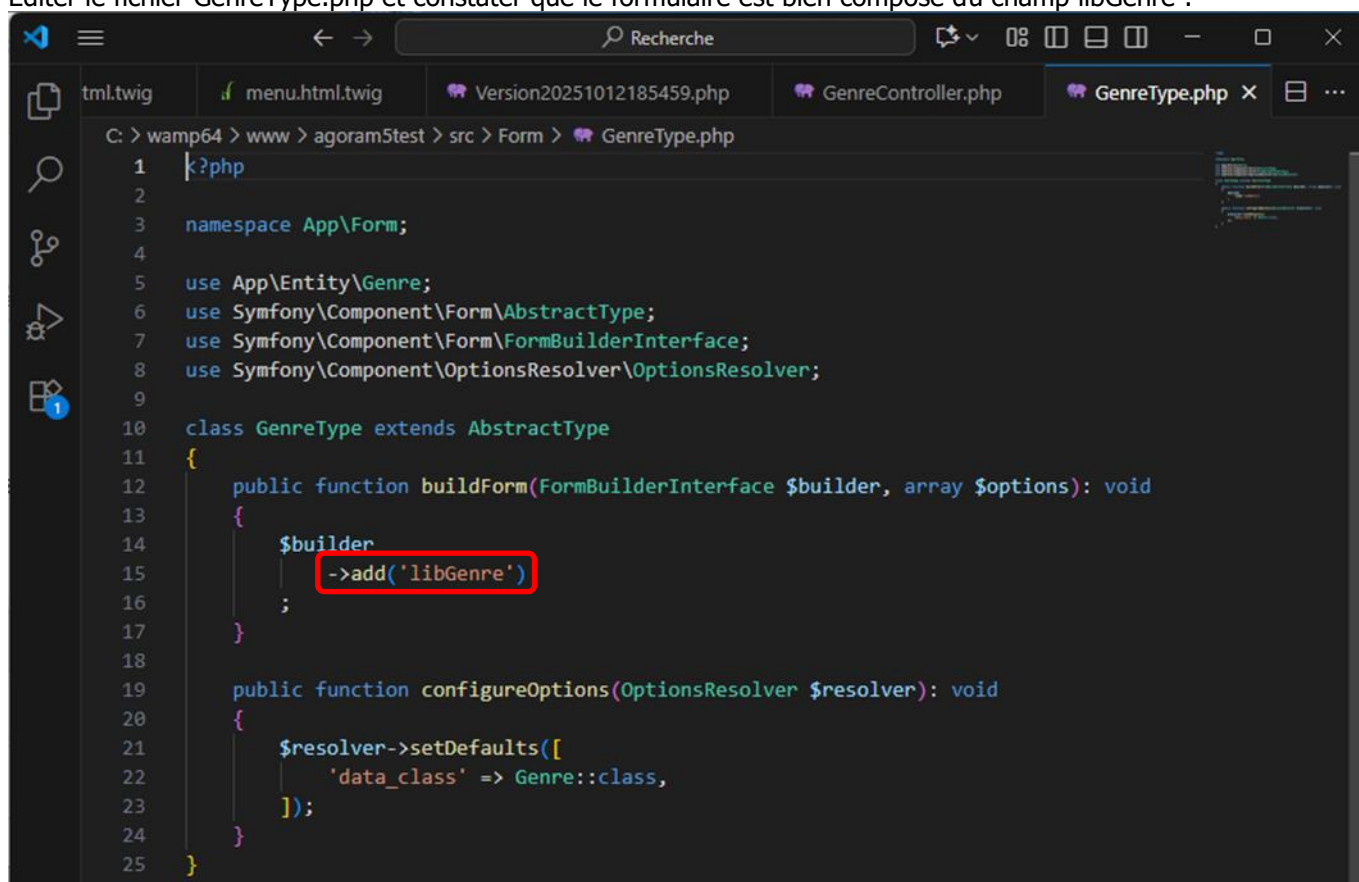
Vérifier, dans ce fichier, l'existence des routes :

/genre	qui lance la vue	genre/index.html.twig
/genre/new	qui lance la vue	genre/new.html.twig
/genre/{id}	qui lance la vue	genre/show.html.twig
/genre/{id}/edit	qui lance la vue	genre/edit.html.twig
et une dernière variante de /genre/{id}		qui se charge de la suppression

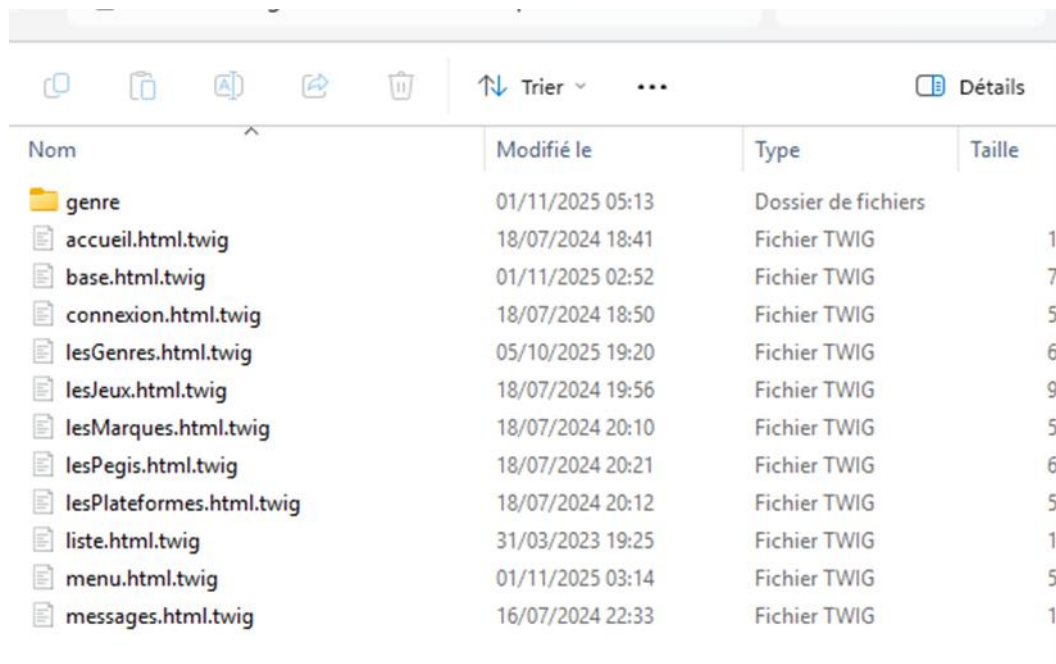
Vérifier la création du dossier « form » et du fichier GenreType.php qui contient la description des champs du formulaire :



Editer le fichier GenreType.php et constater que le formulaire est bien composé du champ libGenre :

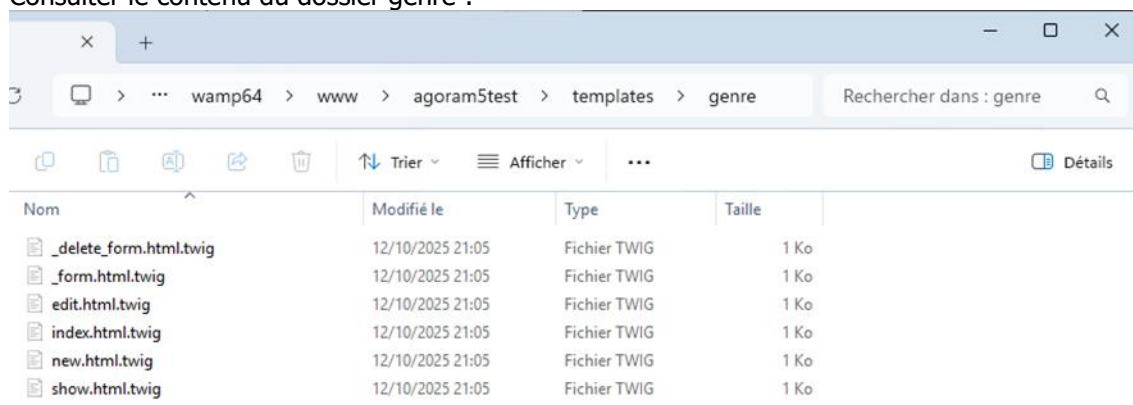


Vérifier la création du dossier « genre » dans le dossier template qui contient les fichiers de vue en TWIG.



Nom	Modifié le	Type	Taille
genre	01/11/2025 05:13	Dossier de fichiers	
accueil.html.twig	18/07/2024 18:41	Fichier TWIG	1
base.html.twig	01/11/2025 02:52	Fichier TWIG	7
connexion.html.twig	18/07/2024 18:50	Fichier TWIG	5
lesGenres.html.twig	05/10/2025 19:20	Fichier TWIG	6
lesJeux.html.twig	18/07/2024 19:56	Fichier TWIG	9
lesMarques.html.twig	18/07/2024 20:10	Fichier TWIG	5
lesPegis.html.twig	18/07/2024 20:21	Fichier TWIG	6
lesPlateformes.html.twig	18/07/2024 20:12	Fichier TWIG	5
liste.html.twig	31/03/2023 19:25	Fichier TWIG	1
menu.html.twig	01/11/2025 03:14	Fichier TWIG	5
messages.html.twig	16/07/2024 22:33	Fichier TWIG	1

Consulter le contenu du dossier genre :



Nom	Modifié le	Type	Taille
_delete_form.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko
_form.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko
edit.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko
index.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko
new.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko
show.html.twig	12/10/2025 21:05	Fichier TWIG	1 Ko

Editer l'un des fichiers de vue TWIG :

```

1  {% extends 'base.html.twig' %}
2
3  {% block title %}Genre{% endblock %}
4
5  {% block body %}
6      <h1>Genre</h1>
7
8      <table class="table">
9          <tbody>
10             <tr>
11                 <th>Id</th>
12                 <td>{{ genre.id }}</td>
13             </tr>
14             <tr>
15                 <th>LibGenre</th>
16                 <td>{{ genre.libGenre }}</td>
17             </tr>
18         </tbody>
19     </table>
20
21     <a href="{{ path('app_genre_index') }}">back to list</a>
22
23     <a href="{{ path('app_genre_edit', {'id': genre.id}) }}">edit</a>
24
25     {{ include('genre/_delete_form.html.twig') }}
26 {% endblock %}

```

2.6 Tests

Lancer le serveur local Symfony :

```

C:\wamp64\www\agoram5test>symfony serve --no-tls
Following Web Server log file (C:\Users\narbe\.symfony5\log\1ef13992078adfb2f7ffdb1889a66d01f23492e0.log)
Following PHP-CGI log file (C:\Users\narbe\.symfony5\log\1ef13992078adfb2f7ffdb1889a66d01f23492e0\79ca75f9e90b4126a5955a33ea6a41ec5e854698.log)

[WARNING] The local web server is optimized for local development and MUST never be used in a production setup.

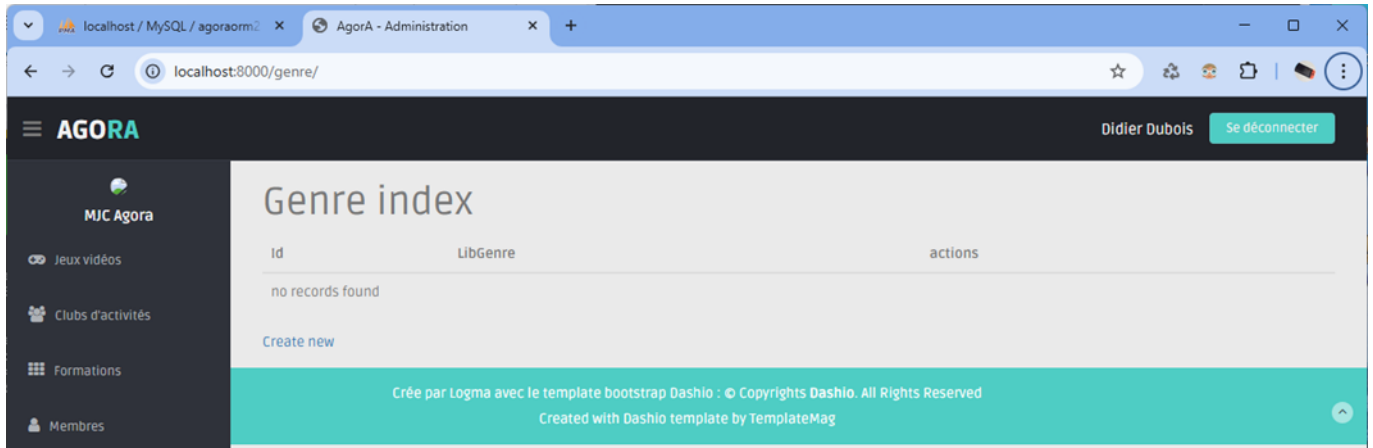
[WARNING] Please note that the Symfony CLI only listens on 127.0.0.1 by default since version 5.10.3.
You can use the --allow-all-ip or --listen-ip flags to change this behavior.

[OK] Web server listening
The Web server is using PHP CGI 8.2.26
http://127.0.0.1:8000

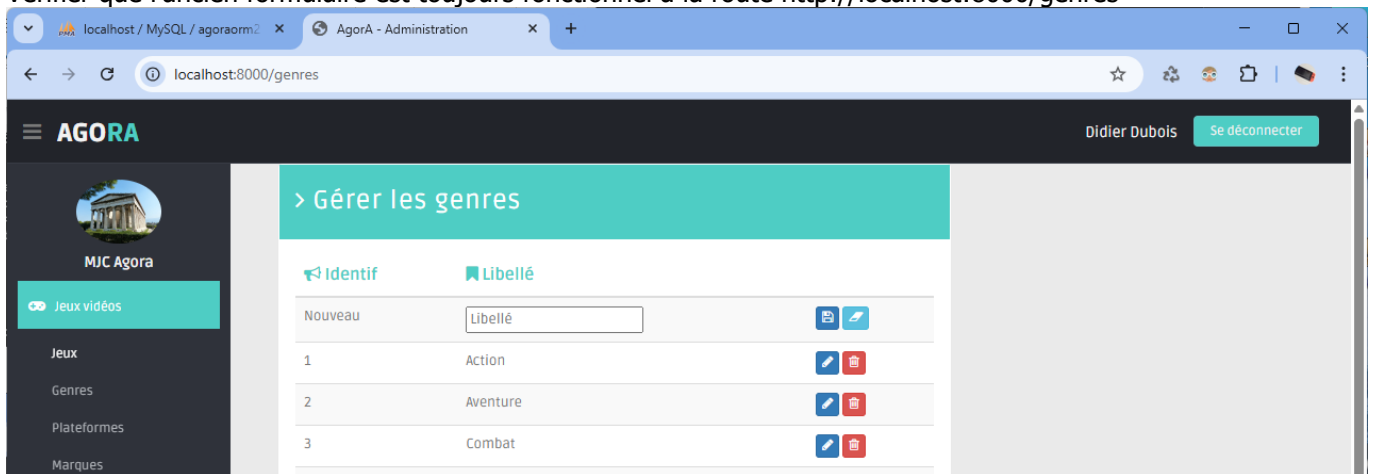
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing query: SELECT DATABASE() sql="SELECT DATABASE()"
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing statement: SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = ? AND TABLE_TYPE = 'BASE TABLE' ORDER BY TABLE_NAME (parameters: array{"1":"agoraorm2"}, types: array{"1":2}) params={"1":"agoraorm2"} sql="SELECT TABLE_NAME\nFROM information_schema.TABLES\nWHERE TABLE_SCHEMA = ?\n AND TABLE_TYPE = 'BASE TABLE'\nORDER BY TABLE_NAME" types={"1":2}
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing statement: CREATE TABLE doctrine_migration_versions (version VARCHAR(191) NOT NULL, executed_at DATETIME DEFAULT NULL, execution_time INT DEFAULT NULL, PRIMARY KEY(version)) DEFAULT CHARACTER SET utf8 COLLATE 'utf8_unicode_ci' ENGINE = InnoDB sql="CREATE TABLE doctrine_migration_versions (version VARCHAR(191) NOT NULL, executed_at DATETIME DEFAULT NULL, execution_time INT DEFAULT NULL, PRIMARY KEY(version)) DEFAULT CHARACTER SET utf8 COLLATE 'utf8_unicode_ci' ENGINE = InnoDB"
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing query: SELECT * FROM doctrine_migration_versions sql="SELECT * FROM doctrine_migration_versions"
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing query: SELECT * FROM doctrine_migration_versions
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Executing query: SELECT * FROM doctrine_migration_versions
[Application] Oct 12 18:58:49 |DEBUG | DOCTRINE Beginning transaction

```

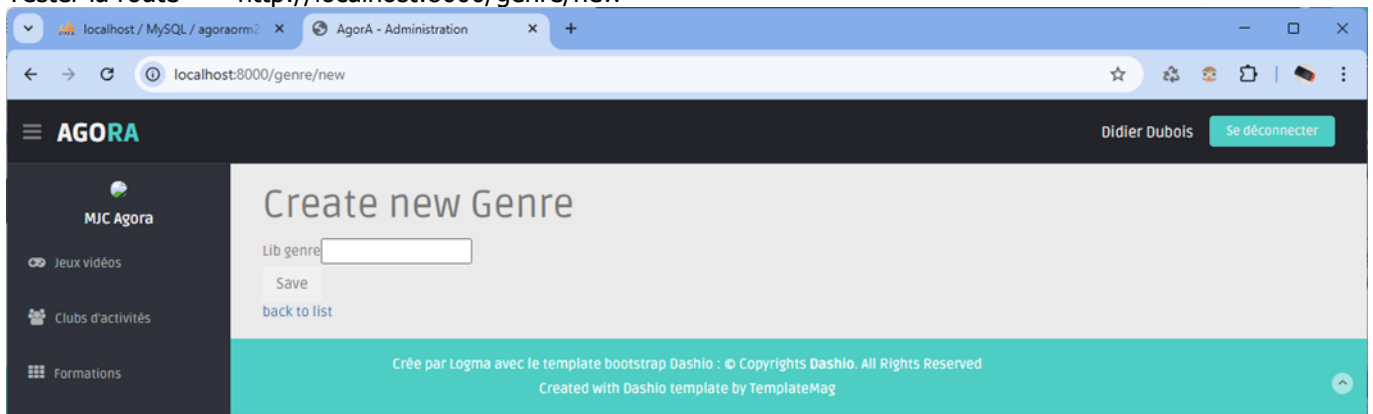
Tester la route <http://localhost:8000/genre> :



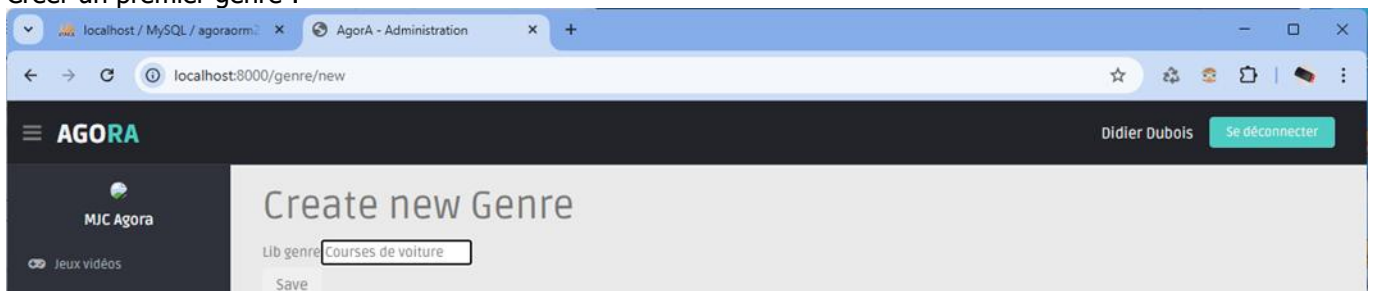
Vérifier que l'ancien formulaire est toujours fonctionnel à la route <http://localhost:8000/genres>

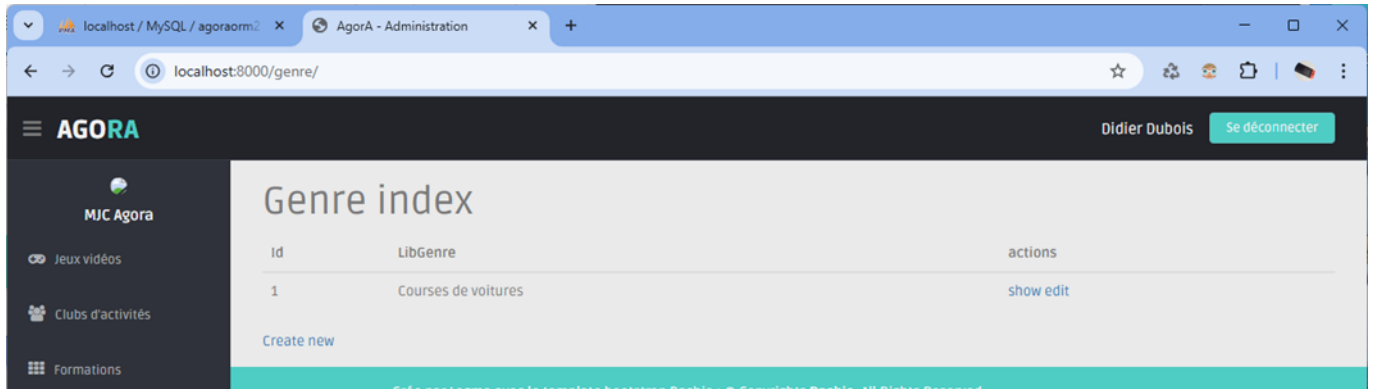


Tester la route <http://localhost:8000/genre/new>

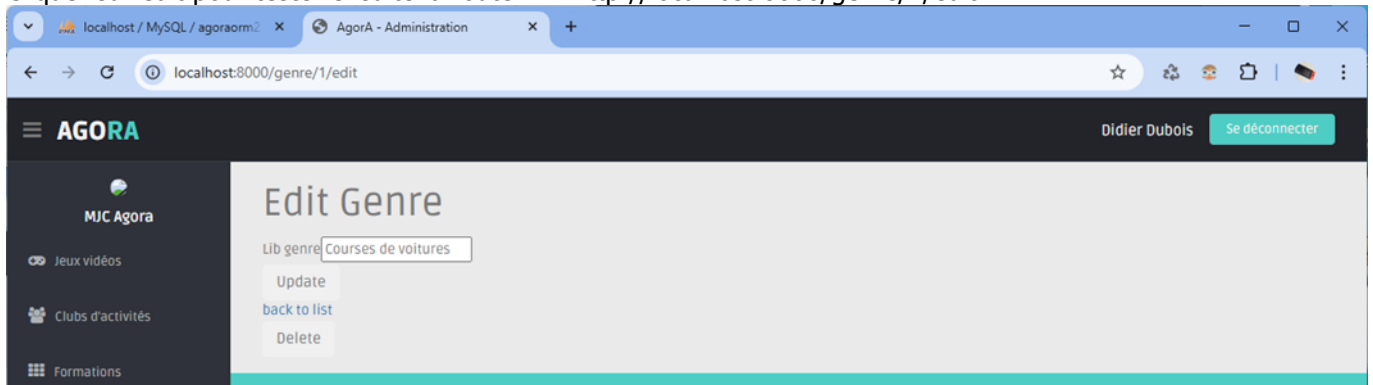


Créer un premier genre :

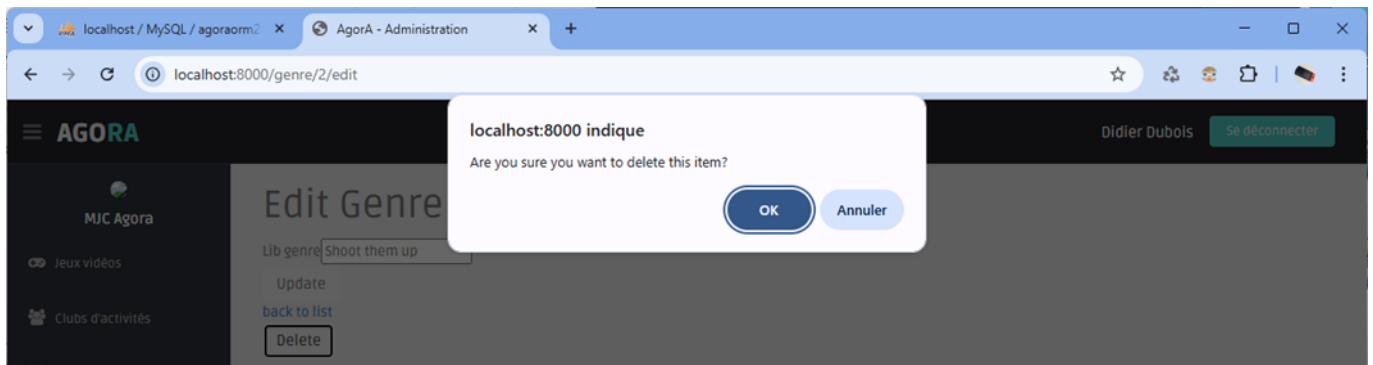




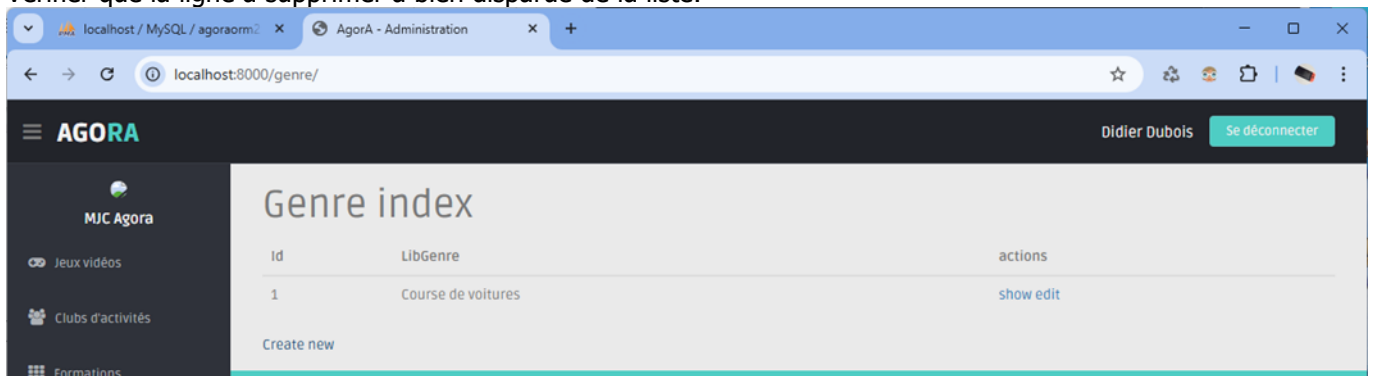
Cliquer sur edit pour tester ensuite la route <http://localhost:8000/genre/1/edit>



Cliquer sur « delete » pour tester la route <http://localhost:8000/genre/2/delete>

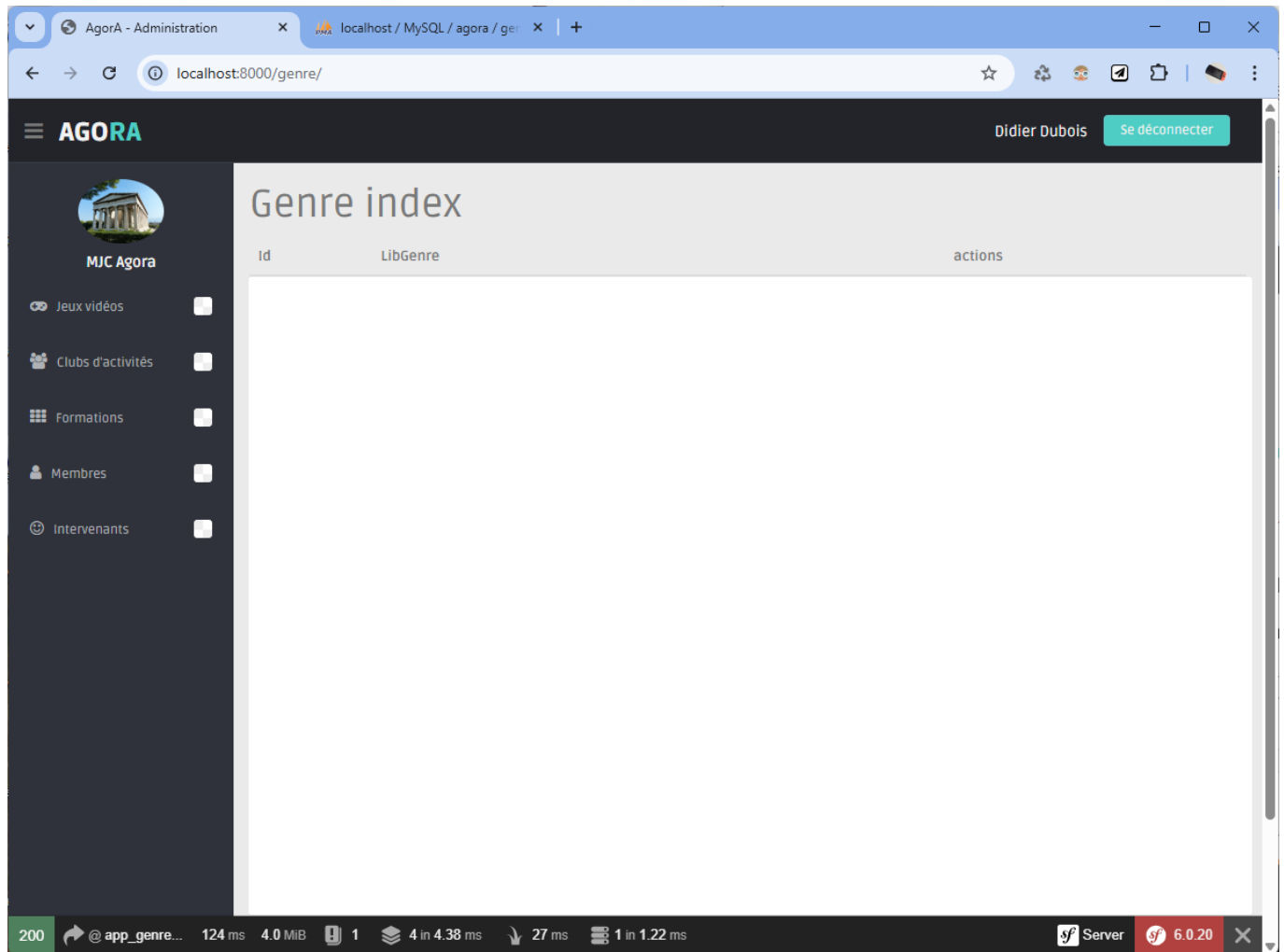


Vérifier que la ligne à supprimer a bien disparue de la liste.



2.7 Ajustement du menu pour appeler le nouveau controleur

Remarquer que le menu jeux vidéos se retracte quand on clique sur l'entrée « Nouvelle page genre »



Modifier l'appel de la vue principale dans le `src/controller/GenreController.php` pour laisser le menu actif

```
<?php

namespace App\Controller;

use App\Entity\Genre;
use App\Form\GenreType;
use App\Repository\GenreRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/genre')]
class GenreController extends AbstractController
{
    #[Route('/', name: 'app_genre_index', methods: ['GET'])]
    public function index(GenreRepository $genreRepository): Response
    {
        return $this->render('genre/index.html.twig', [
            'genres' => $genreRepository->findAll(), 'menuActif' => 'Jeux',
        ]);
    }
}
```

Faire un test en cliquant sur "Genres" dans le menu "Jeux vidéos" de l'application et vérifier que l'on obtient l'affichage suivant et que le menu "Jeux vidéos" reste actif après le clic.

The screenshot shows a web browser window with two tabs: 'localhost / MySQL / agoraorm2' and 'AgorA - Administration'. The address bar shows 'localhost:8000/genre/'. The application header is dark blue with the 'AGORA' logo on the left and the user 'Didier Dubois' with a 'Se déconnecter' button on the right. A left sidebar contains a menu with 'MJC Agora' at the top, followed by 'Jeux vidéos' (highlighted in red), 'Jeux', 'Genres', 'Plateformes', and 'Marques'. The main content area is titled 'Genre index' and contains a table with the following data:

Id	LibGenre	actions
1	Course de voitures	show edit

Below the table is a 'Create new' link. At the bottom of the page, a red footer contains the text: 'Crée par Logma avec le template bootstrap Dashio : © Copyrights Dashio. All Rights Reserved' and 'Created with Dashio template by TemplateMag'.

3. Modification du style des nouveaux formulaires

3.1 Modification du template genre/index.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/index.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Genre index{% endblock %}

{% block body %}
    <h1>Genre index</h1>

    <table class="table">
        <thead>
            <tr>
                <th>Id</th>
                <th>LibGenre</th>
                <th>actions</th>
            </tr>
        </thead>
        <tbody>
            {% for genre in genres %}
                <tr>
                    <td>{{ genre.id }}</td>
                    <td>{{ genre.libGenre }}</td>
                    <td>
                        <a href="{{ path('app_genre_show',{'id': genre.id}) }}">show</a>
                        <a href="{{ path('app_genre_edit',{'id': genre.id}) }}">edit</a>
                    </td>
                </tr>
            {% else %}
                <tr>
                    <td colspan="3">no records found</td>
                </tr>
            {% endfor %}
        </tbody>
    </table>

    <a href="{{ path('app_genre_new') }}">Create new</a>
{% endblock %}
```

Modifier le fichier du template genre/index.html.twig pour :

- lui donner un **style visuel similaire aux autres vues** de l'application en utilisant les mêmes feuilles de styles que les autres template,
- **franciser l'affichage**

Fichier agora5/templates/genre/index.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Genre index{% endblock %} #}

{% block body %}
    <!-- page start-->
```

```

<div class="col-sm-8">
  <section class="panel">
    <div class="chat-room-head">
      <h3><i class="fa fa-angle-right"></i> Gérer les genres</h3>
    </div>
    <div class="panel-body">

      {# <h1>Genre index</h1> #}

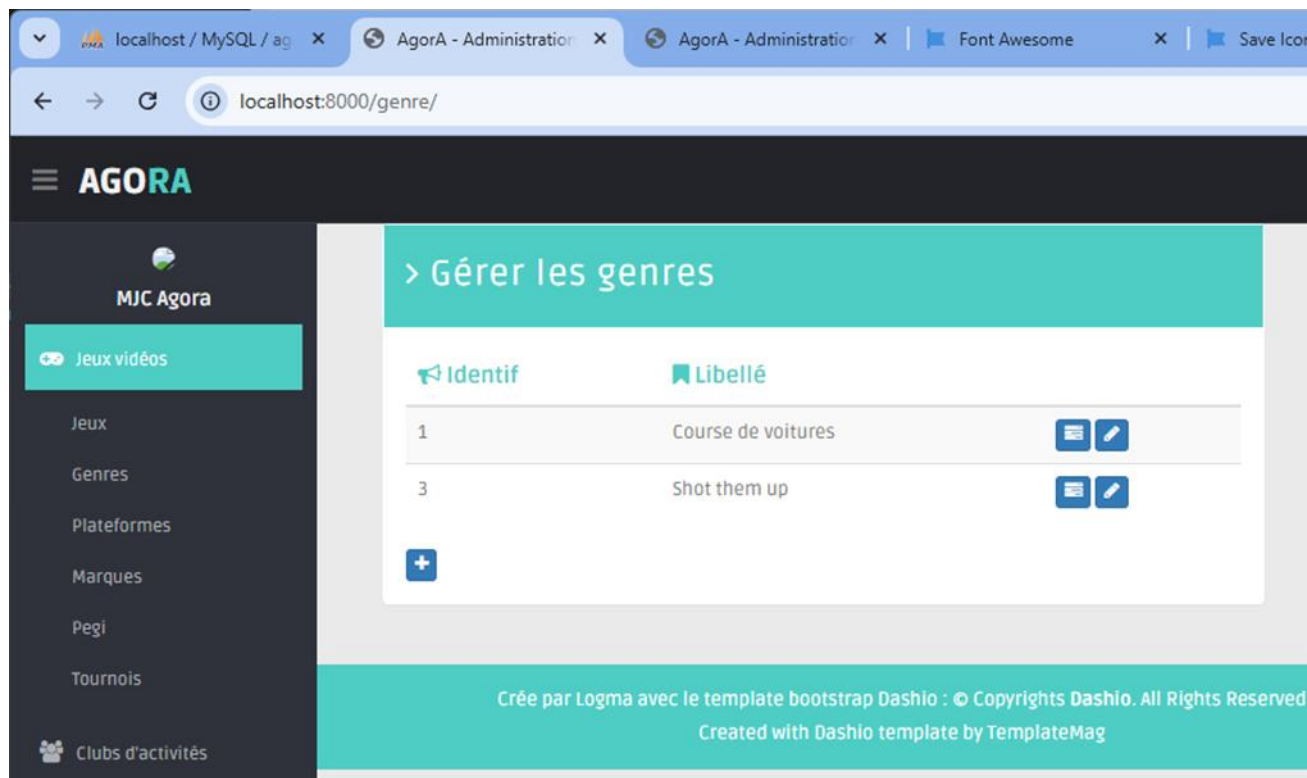
      <table class="table" table-striped table-advance table-hover>
        <thead>
          <tr class="tableau-entete">
            <th><i class="fa fa-bullhorn"></i> Identif</th>
            <th><i class="fa fa-bookmark"></i> Libellé</th>
            <th>actions</th>
          </tr>
        </thead>
        <tbody>
          {% for genre in genres %}
            <tr>
              <td>{{ genre.id }}</td>
              <td>{{ genre.libGenre }}</td>
              <td>
                <a href="{% path('app_genre_show', {'id': genre.id}) %}"><button
class="btn btn-primary btn-xs"><i class="fa fa-tasks"></i></button></a>
                <a href="{% path('app_genre_edit', {'id': genre.id}) %}"><button
class="btn btn-primary btn-xs"><i class="fa fa-pencil"></i></button></a>
              </td>
            </tr>
          {% else %}
            <tr>
              <td colspan="3">aucun genre enregistré en base de données</td>
            </tr>
          {% endfor %}
        </tbody>
      </table>

      <a href="{% path('app_genre_new') %}"><button class="btn btn-primary btn-
xs"><i class="fa fa-plus"></i></button></a>
    </div><!-- fin div panel-body-->
  </section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->

{% endblock %}

```

Cliquer de nouveau sur Genres dans le menu et observer les modifications :



3.2 Modification du template genre/edit.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/edit.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Edit Genre{% endblock %}

{% block body %}
    <h1>Edit Genre</h1>

    {{ include('genre/_form.html.twig', {'button_label': 'Update'}) }}

    <a href="{{ path('app_genre_index') }}">back to list</a>

    {{ include('genre/_delete_form.html.twig') }}
{% endblock %}
```

Modifier le fichier du template genre/edit.html.twig pour :

- lui donner un **style visuel similaire aux autres vues** de l'application en utilisant les mêmes feuilles de styles que les autres template,
- **franciser l'affichage**

Fichier agora5/templates/genre/edit.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Edit Genre{% endblock %}

{% block body %}
    <!-- page start -->
    <div class="col-sm-8">
        <section class="panel">
```

```

<div class="chat-room-head">
  <h3><i class="fa fa-angle-right"></i> Editer un genre</h3>
</div>
<div class="panel-body">

  <h1>Edit Genre</h1>

  {{ include('genre/_form.html.twig', {'button_label': 'Enregistrer'}) }}

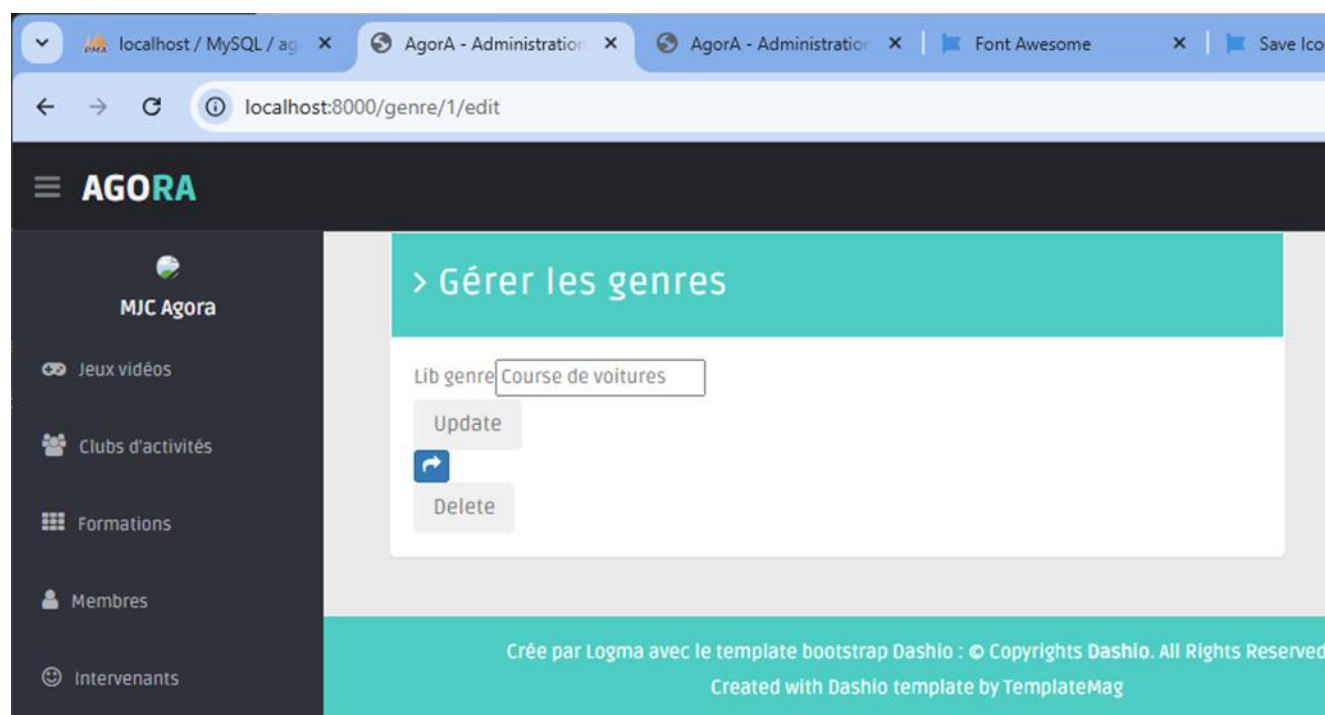
  <a href="{{ path('app_genre_index') }}"><button class="btn btn-primary btn-
xs"><i class="fa fa-share"></i></button></a>

  {{ include('genre/_delete_form.html.twig') }}
  </div><!-- fin div panel-body-->
</section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->

{% endblock %}

```

Cliquer de nouveau sur Genres dans le menu et observer les modifications :



3.3 Modification du template genre/_delete_form.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/_delete_form.html.twig

```

<form method="post" action="{{ path('app_genre_delete', {'id': genre.id}) }}"
onsubmit="return confirm('Are you sure you want to delete this item?');">
  <input type="hidden" name="_token" value="{{ csrf_token('delete' ~ genre.id) }}">
  <button class="btn">Delete</button>
</form>

```

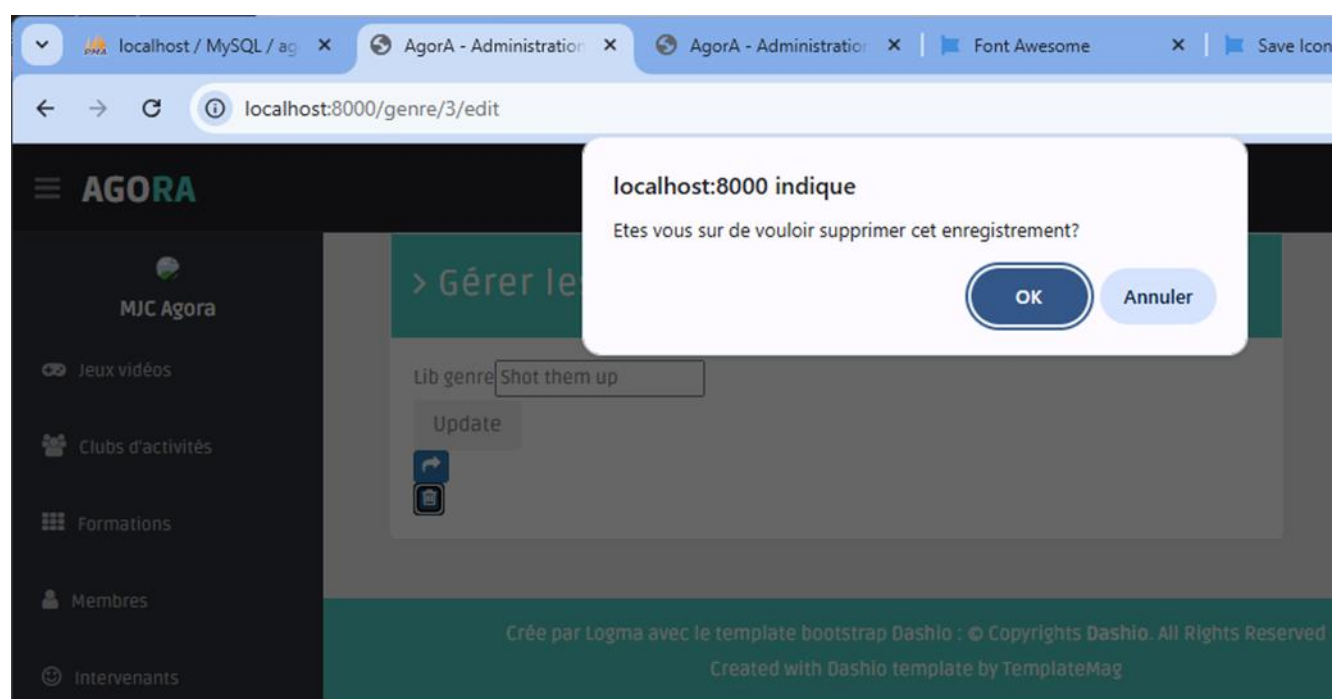
Modifier le fichier du template genre/_delete_form.html.twig pour :

- lui donner un style visuel similaire aux autres vues de l'application en utilisant les mêmes feuilles de styles que les autres template,
- franciser l'affichage

Fichier agora5/templates/genre/_delete_form.html.twig

```
<form method="post" action="{% path('app_genre_delete', {'id': genre.id}) %}"
onsubmit="return confirm('Etes vous sur de vouloir supprimer cet
enregistrement?');">
  <input type="hidden" name="_token" value="{% csrf_token('delete' ~ genre.id)
  %}">
  <button class="btn btn-primary btn-xs"><i class="fa fa-trash-o"></i></button>
</form>
```

Cliquer de nouveau sur « supprimer » et observer les modifications :



3.4 Modification du template genre/new.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/new.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}New Genre{% endblock %}

{% block body %}
  <h1>Create new Genre</h1>

  {{ include('genre/_form.html.twig') }}

  <a href="{% path('app_genre_index') %}">back to list</a>
{% endblock %}
```

Modifier le fichier du template genre/new.html.twig pour :

- lui donner un **style visuel similaire aux autres vues** de l'application en utilisant les mêmes feuilles de styles que les autres template,
- **franciser l'affichage**

Fichier agora5/templates/genre/new.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}New Genre{% endblock %}

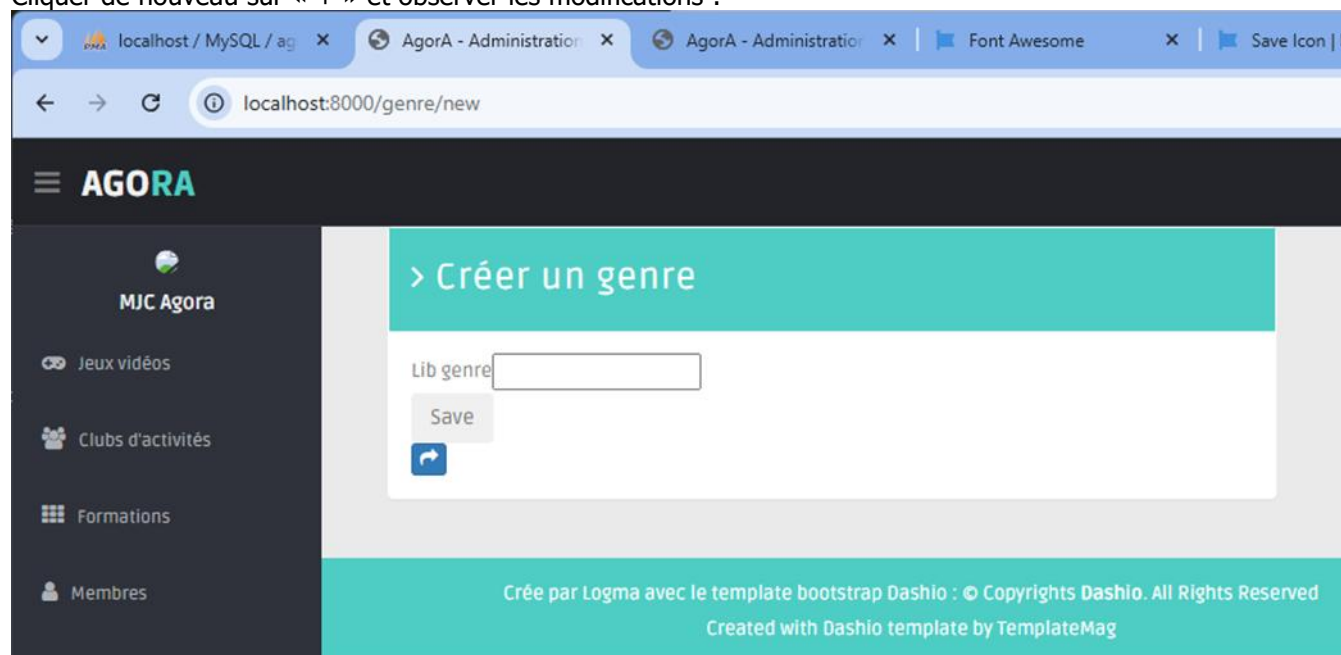
{% block body %}
<!-- page start-->
<div class="col-sm-8">
  <section class="panel">
    <div class="chat-room-head">
      <h3><i class="fa fa-angle-right"></i> Créer un genre</h3>
    </div>
    <div class="panel-body">

      <h1>Create new Genre</h1>

      {{ include('genre/_form.html.twig') }}

      <a href="{{ path('app_genre_index') }}"><button class="btn btn-primary btn-
xs"><i class="fa fa-share"></i></button></a>
    </div><!-- fin div panel-body-->
  </section><!-- fin section tournois-->
</div><!-- fin div col-sm-8-->
{% endblock %}
```

Cliquer de nouveau sur « + » et observer les modifications :



3.5 Modification du template genre/_form.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/_form.html.twig


```

{{ form_start(form) }}
    {{ form_widget(form) }}
    <button class="btn">{{ button_label|default('Save') }}</button>
{{ form_end(form) }}

```

Modifier le fichier du template genre/_form.html.twig pour :

- lui donner un **style visuel similaire aux autres vues** de l'application en utilisant les mêmes feuilles de styles que les autres template,
- **franciser l'affichage**

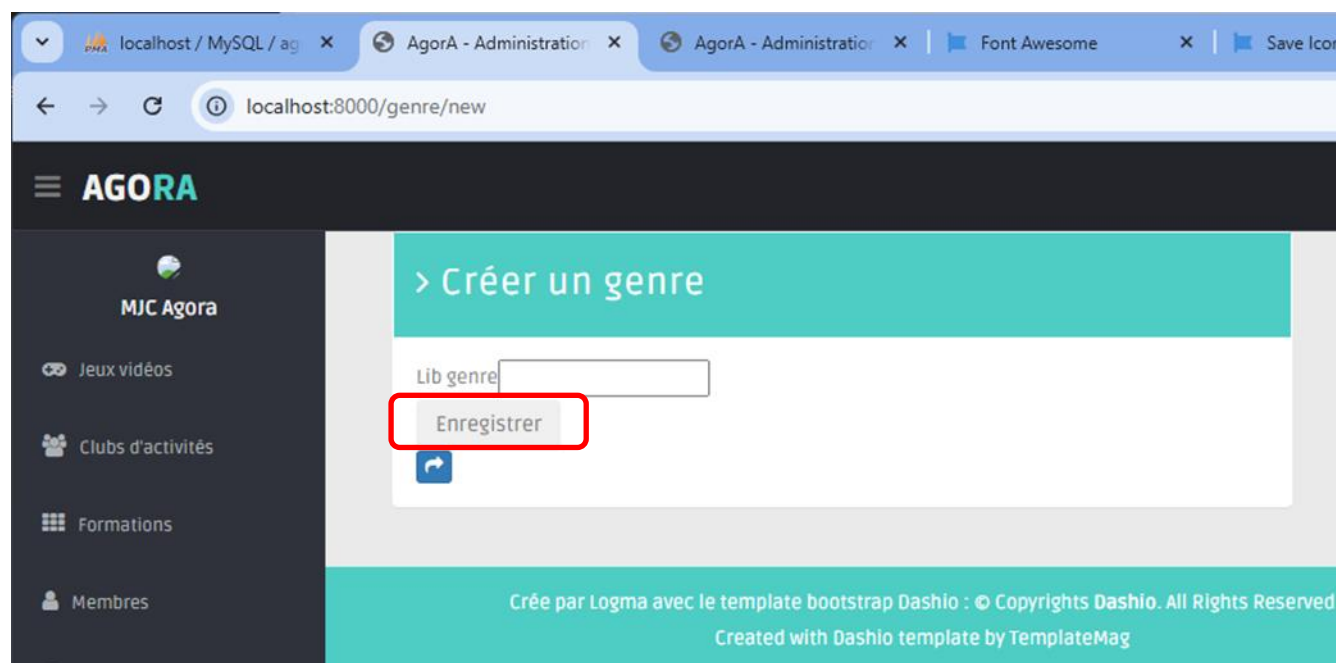
Fichier agora5/templates/genre/_form.html.twig

```

{{ form_start(form) }}
    {{ form_widget(form) }}
    <button class="btn">{{ button_label|default('Enregistrer') }}</button>
{{ form_end(form) }}

```

Cliquer de nouveau sur « + » et observer les modifications :



3.6 Modification du template genre/show.html.twig :

Ci-dessous le template généré automatiquement par Symfony qu'il va falloir adapter à notre contexte.

Fichier agoram5/templates/genre/show.html.twig

```

{% extends 'base.html.twig' %}

{% block title %}Genre{% endblock %}

{% block body %}
    <h1>Genre</h1>

    <table class="table">
        <tbody>
            <tr>
                <th>Id</th>
                <td>{{ genre.id }}</td>
            </tr>
        </tbody>
    </table>

```

```

        </tr>
        <tr>
            <th>LibGenre</th>
            <td>{{ genre.libGenre }}</td>
        </tr>
    </tbody>
</table>

<a href="{{ path('app_genre_index') }}">back to list</a>

<a href="{{ path('app_genre_edit', {'id': genre.id}) }}">edit</a>

{{ include('genre/_delete_form.html.twig') }}
{% endblock %}

```

Modifier le fichier du template genre/show.html.twig pour :

- lui donner un **style visuel similaire aux autres vues** de l'application en utilisant les mêmes feuilles de styles que les autres template,
- **franciser l'affichage**

Fichier agora5/templates/genre/show.html.twig

```

{% extends 'base.html.twig' %}

{% block title %}Genre{% endblock %}

{% block body %}
<!-- page start -->
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Créer un genre</h3>
        </div>
        <div class="panel-body">

            <h1>Genre</h1>

            <table class="table table-striped table-advance table-hover">
                <tbody>
                    <tr>
                        <th class="tableau-entete"><i class="fa fa-bullhorn"></i>
Identif</th>
                        <td>{{ genre.id }}</td>
                    </tr>
                    <tr>
                        <th class="tableau-entete"><i class="fa fa-bookmark"></i>
Libellé</th>
                        <td>{{ genre.libGenre }}</td>
                    </tr>
                </tbody>
            </table>

            <a href="{{ path('app_genre_index') }}"><button class="btn btn-primary btn-
xs"><i class="fa fa-share"></i></button></a>

```

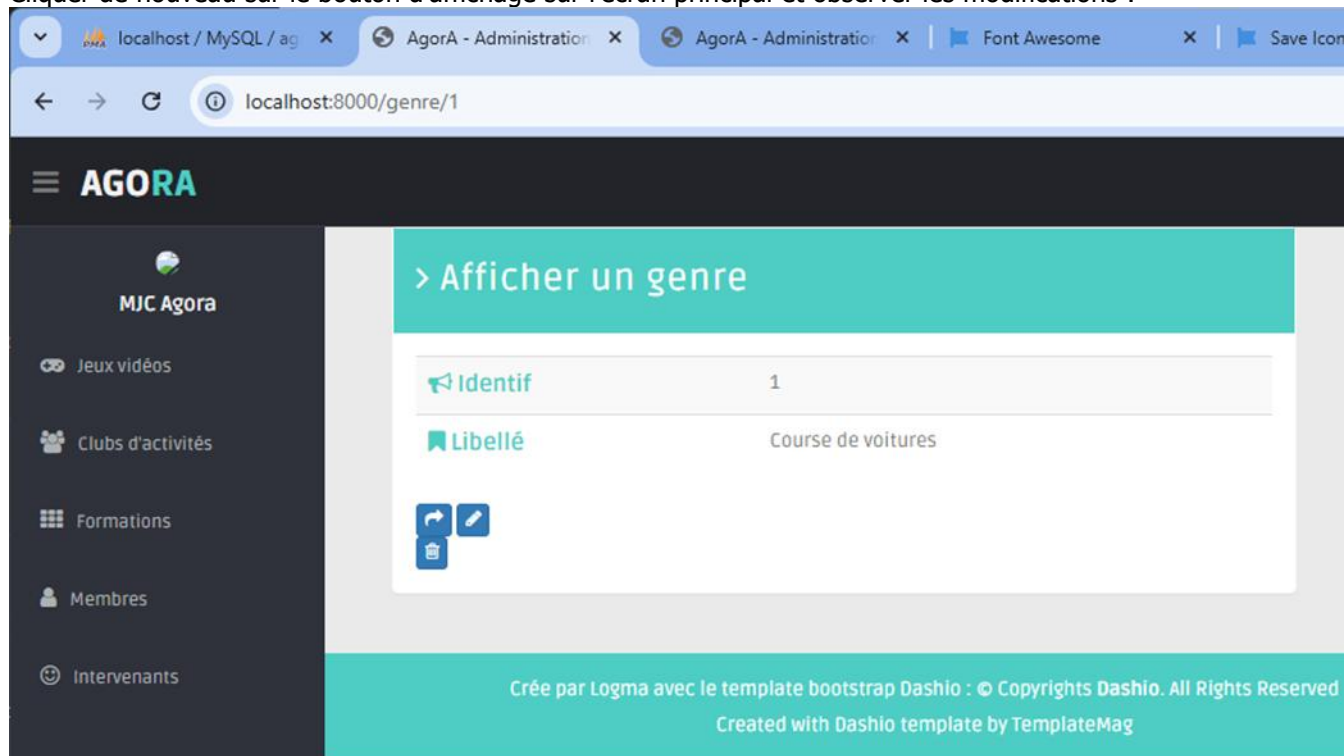
```

<a href="{{ path('app_genre_edit', {'id': genre.id}) }}"><button class="btn
btn-primary btn-xs"><i class="fa fa-pencil"></i></button></a>

{{ include('genre/_delete_form.html.twig') }}
</div><!-- fin div panel-body-->
</section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->
{% endblock %}

```

Cliquer de nouveau sur le bouton d'affichage sur l'écran principal et observer les modifications :



Ajouter un paramètre dans l'appel de les vues genre/edit.html.twig, genre/show.html.twig, genre/new.html.twig, pour que la vue menu.html.twig la reçoive et que le menu jeux vidéo soit maintenu ouvert.

Fichier agora5/src/Controller/GenreController.php

```

...
#[Route('/new', name: 'app_genre_new', methods: ['GET', 'POST'])]
public function new(Request $request, EntityManagerInterface $entityManager):
Response
{
    $genre = new Genre();
    $form = $this->createForm(GenreType::class, $genre);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->persist($genre);
        $entityManager->flush();

        return $this->redirectToRoute('app_genre_index', [],
Response::HTTP_SEE_OTHER);
    }

    return $this->renderForm('genre/new.html.twig', [

```

```

        'genre' => $genre,
        'form' => $form,
        'menuActif' => 'Jeux',
    ]);
}

#[Route('/{id}', name: 'app_genre_show', methods: ['GET'])]
public function show(Genre $genre): Response
{
    return $this->render('genre/show.html.twig', [
        'genre' => $genre,
        'menuActif' => 'Jeux',
    ]);
}

#[Route('/{id}/edit', name: 'app_genre_edit', methods: ['GET', 'POST'])]
public function edit(Request $request, Genre $genre, EntityManagerInterface
$entityManager): Response
{
    $form = $this->createForm(GenreType::class, $genre);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->flush();

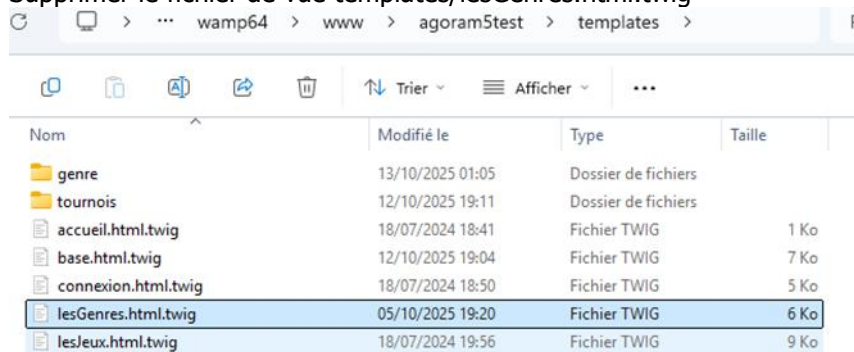
        return $this->redirectToRoute('app_genre_index', [],
Response::HTTP_SEE_OTHER);
    }

    return $this->renderForm('genre/edit.html.twig', [
        'genre' => $genre,
        'form' => $form,
        'menuActif' => 'Jeux',
    ]);
}

```

3.7 Suppression de l'ancien CRUD de gestion des genres

Supprimer le fichier de vue templates/lesGenres.html.twig



Nom	Modifié le	Type	Taille
genre	13/10/2025 01:05	Dossier de fichiers	
tournois	12/10/2025 19:11	Dossier de fichiers	
accueil.html.twig	18/07/2024 18:41	Fichier TWIG	1 Ko
base.html.twig	12/10/2025 19:04	Fichier TWIG	7 Ko
connexion.html.twig	18/07/2024 18:50	Fichier TWIG	5 Ko
lesGenres.html.twig	05/10/2025 19:20	Fichier TWIG	6 Ko
lesJeux.html.twig	18/07/2024 19:56	Fichier TWIG	9 Ko

Supprimer le contrôleur src/Controller/GenresController.php

Nom	Modifié le	Type	Taille
modele	12/10/2025 17:20	Dossier de fichiers	
.gitignore	16/07/2024 18:18	Fichier source Git I...	0 Ko
AccueilController.php	16/07/2024 22:54	Fichier source PHP	1 Ko
ConnexionController.php	16/07/2024 23:06	Fichier source PHP	2 Ko
GenreController.php	12/10/2025 23:50	Fichier source PHP	3 Ko
GenresController.php	05/10/2025 19:21	Fichier source PHP	4 Ko
JeuxController.php	18/07/2024 20:00	Fichier source PHP	4 Ko

Supprimer les méthodes ajouterGenre(), modifierGenre(), supprimerGenre() dans le fichier de modele class.PdoJeux.inc.php (conserver la méthode getLesGenres()) utilisée dans le controleur de gestion des jeux).

```

C: > wamp64 > www > agoram5test > src > Controller > modele > class.PdoJeux.inc.php
20 {
118
119
120     /**
121     * Ajoute un nouveau genre avec le libellé et le spécialiste passé en paramètre
122     *
123     * @param string $libGenre : le libellé du genre à ajouter
124     * @return int l'identifiant du genre créé
125     */
126     public function ajouterGenre(string $libGenre): int
127     // public function ajouterGenre(string $libGenre, int $idSpecialiste): int
128     {
129         // $requete = "INSERT INTO genre (idGenre, libGenre, idSpecialiste)
130         // VALUES (0, :unLibGenre, :unIdSpecialiste)";
131         $requete = "INSERT INTO genre (idGenre, libGenre)
132         VALUES (0, :unLibGenre)";
133         try {
134             $requete_prepare = PdoJeux::$monPdo->prepare($requete);
135             $requete_prepare->bindParam(':unLibGenre', $libGenre, PDO::PARAM_STR);
136             // $requete_prepare->bindParam(':unIdSpecialiste', $idSpecialiste, PDO::PARAM_INT);
137             $requete_prepare->execute();
138             // récupérer l'identifiant créé
139             return PdoJeux::$monPdo->lastInsertId();
140         } catch (Exception $e) {
141             die('<div class = "erreur">Erreur dans la requête !<p>'. $e->getMessage() . '</p>');
142         }
143     }
144
145     /**
146     * Modifie le libellé et le spécialiste du genre passé en paramètre
147     *
148     * @param int $idGenre : l'identifiant du genre à modifier
149     * @param string $libGenre : le libellé modifié
150     */
151     public function modifierGenre(int $idGenre, string $libGenre): void
152     // public function modifierGenre(int $idGenre, string $libGenre, int $idSpecialiste): v
153     {
154         // $requete = "UPDATE genre SET libGenre = :unLibGenre, idSpecialiste = :unIdSpecialiste
155         $requete = "UPDATE genre SET libGenre = :unLibGenre
  
```

Supprimer l'entrée dans templates/menu.html.twig correspondant à cet ancien formulaire et ajuster le nom de l'entrée qui lance le nouveau formulaire :

```

18 <ul class="sub">
19 <li><a href="{{path('jeux_afficher')}}">Jeux</a></li>
20 <li><a href="{{path('genres_afficher')}}">Genres</a></li>
21 <li><a href="{{path('app_genre_index')}}">Nouvelle page Genres</a></li>
22 <li><a href="{{path('plateformes_afficher')}}">Plateformes</a></li>
23 <li><a href="{{path('marques_afficher')}}">Marques</a></li>
24 <li><a href="{{path('pegis_afficher')}}">Pegi</a></li>
25 </ul>
  
```



```
18 <ul class="sub">
19 <li><a href="{{path('jeux_afficher')}}">Jeux</a></li>
20 <li><a href="{{path('app_genre_index')}}">Genres</a></li>
21 <li><a href="{{path('plateformes_afficher')}}">Plateformes</a></li>
22 <li><a href="{{path('marques_afficher')}}">Marques</a></li>
23 <li><a href="{{path('pegis_afficher')}}">Pegi</a></li>
24 </ul>
```

4. A vous de jouer

Sur le modèle du contrôleur permettant de gérer les genres, répartissez-vous le travail au sein de votre équipe projet pour générer les entités puis les contrôleurs permettant de gérer les PEGIs, les plateformes et les marques.

Le formulaire permettant de gérer les jeux sera traité ultérieurement.